

Where Do the Experts Go?

Bounding, Measuring and Closing the Gap for Mixture-of-Experts Decode with Experts in Host Memory

Harshith Kantamneni
Independent Researcher

Abstract

A large Mixture-of-Experts (MoE) model runs on a desktop GPU only if most of its experts stay in host memory. Many systems propose where those experts should execute; few are measured against a ceiling, and the most widely used engine, llama.cpp, simply assigns whole layers' experts to the CPU. We contribute four things. **(1)** Exact decode-time routing traces recorded on a CPU-only machine, by streaming teacher-forced prefill one decoder layer at a time. **(2)** A bytes-over-bandwidth decode model, validated on 52 published measurements, and a speed-of-light bound for any placement policy with a per-layer GPU budget. **(3)** A pre-registered first-party study on an NVIDIA A10 that separates what the model gets right (decode time is affine in the number of CPU layers, $R^2 \geq 0.996$) from what it does not (constants transferred from third-party data), and that finds three costs hidden in hybrid CPU-GPU decode: expert copies that queue behind the step's own transfers on the copy engine, a host-driven hand-off of 84 μ s per layer paid even when the CPU has nothing to do, and a scheduler that never actually overlapped CPU and GPU work. **(4)** A system in llama.cpp that removes them: a per-layer GPU expert cache with decayed-frequency admission, copies by a kernel that reads pinned memory, and a *mailbox* hand-off in which the GPU itself signals CPU helper threads through pinned memory and waits for them on the device, so that a layer whose experts are all resident never involves the host. In a pre-registered test on fresh sequences, on gpt-oss-20b, gpt-oss-120b and Qwen3-30B-A3B at two quantisations, it decodes $1.41\text{--}1.81\times$ faster than llama.cpp's layer offload at equal GPU memory, including gpt-oss-120b at 72.1 tok/s on a 24 GB GPU, with teacher-forced NLL within 0.6%. The mailbox alone makes llama.cpp's own layer offload $1.02\text{--}1.32\times$ faster. A step-time model fitted only on calibration sequences predicted the test throughput with 2.9% median error.

1 Introduction

A 120B-parameter MoE with 4-bit experts needs 61 GB for its experts alone, while a consumer GPU has 8-32 GB. Running such a model locally means splitting the experts between GPU memory and host DRAM, and batch-1

decode speed is then set largely by where each routed expert's weights are when the router selects it.

The literature offers many mechanisms: demand-fetching into a GPU cache [5], activation-aware caching and prefetching [19, 15, 10, 24, 18], running misses on the CPU [12, 13, 17, 9, 23], and substitute or lower-precision experts [16, 22]. In practice, desktop users run llama.cpp [6], which places whole layers' experts on the CPU (`--n-cpu-moe`). Trace studies disagree on basic questions, such as whether prediction beats LRU [21, 14, 18], and hit rates are a poor proxy for speed once misses can run elsewhere [21].

This paper asks three questions in order. *How fast could batch-1 decode be* for a given model, GPU budget and platform? *Where does the time actually go* in the engine people use? And *how much of the gap can a practical system close*, without changing the model's outputs? We answer the first with traces, a validated model and a bound; the second with a pre-registered study on real hardware; and the third with a system built into llama.cpp and evaluated under a second pre-registration.

Contributions.

- **Exact traces without a GPU** (Section 3); **a validated decode model and a speed-of-light bound** (Section 4), held out source by source on third-party data.
- **A pre-registered first-party study** (Section 5): the model's structure holds on an A10, its transferred constants do not; and a diagnosis of hybrid decode (Section 6) that attributes the missing speed to three implementation costs rather than to bandwidth.
- **A system** (Section 7): an expert cache with CPU-executed misses and a GPU-signalled *mailbox* hand-off, in about 2,000 lines of C++ and CUDA on top of llama.cpp (plus 600 of benchmarks and tests), with logits identical to llama.cpp's own CPU path when the same CPU kernels are used.
- **A pre-registered evaluation** (Section 8): $1.41\text{--}1.81\times$ over llama.cpp at equal GPU memory on four models, fidelity, prediction accuracy of the model, and the fraction of the bound reached. Three of the five registered hypotheses held as registered; the two

that did not (the hand-off-only gain on one model, top-1 agreement on gpt-oss-120b) are reported with their causes.

Code, traces, job scripts, raw measurements and both pre-registrations are released; git history orders every prediction before the measurement it predicts.

2 Background and Related Work

MoE decode moves data. At batch 1 a routed expert is a pair of matrix–vector products whose cost is its weight bytes over the bandwidth of the memory they are read from. The data-movement view of transformers [11] and sparsity as an efficiency lever [7] frame placement as: which bytes cross which link, and how often.

Offloading systems. Demand-paging systems keep a per-layer GPU cache and copy misses over PCIe [5, 19, 15, 16]; prefetchers predict upcoming experts [10, 24, 18]. CPU-execution systems run non-resident experts on the host [12, 13, 17, 9, 23]. MoE-Lightning [3] and MoE-Lens [20] target batched throughput; we target single-user latency. Most of these systems are evaluated against naive offloading or against each other; we compare against llama.cpp’s layer offload, the configuration desktop users actually run, at equal GPU memory and on the same machine.

Caching theory and methodology. Belady’s MIN [1] minimises misses under demand paging, and prefetching interacts with caching [2]. Our admission policy is in the spirit of TinyLFU [4]. We follow Hoeffler and Belli’s guidelines [8]: pre-registered hypotheses, stated statistics, fitted parameters named, and held-out data.

3 Exact Routing Traces Without a GPU

In a causal decoder the hidden state of token t at every layer depends on tokens 1.. t only, so the experts that decode selects for token t are the experts that teacher-forced prefill selects at position t , up to floating-point ties. Prefill lets us invert the loop order: load *one* decoder layer (tensors read from the model hub with HTTP range requests, never stored), push the whole corpus through it, record the router’s top- k , free the layer, move on. Weights are read once per corpus and peak memory is one layer. Against the reference **transformers** implementations of Qwen3-MoE, OLMoE, DeepSeek-V2 and gpt-oss (run both as prefill and as token-by-token decode with a KV cache) all selections match exactly on tiny random checkpoints. The corpus is two-turn conversations from four public datasets (chat, math, code, multilingual), about 6k tokens per domain, each formatted with the model’s own chat

Table 1: Model-term ablation, leave-one-source-out on published measurements.

Variant	params	median	MAPE
M0 bytes/BW + hand-off	4	21%	24%
M1 + per-transfer latency	5	17%	18%
M4 M1 + CPU per-expert lat.	6	16%	17%

template; prompt tokens are prefill and assistant tokens are decode steps, replayed back to back as one user’s session. Everything ran on a 2-vCPU, 7 GB VM. We trace gpt-oss-120b too but do not use its routing: on some conversations its per-token loss climbs to near-uniform partway through the assistant turn, in our fp32 CPU implementation *and*, as Section 8 shows, in llama.cpp; we therefore also evaluate on each model’s own sampled text.

4 A Decode Model and a Speed-of-Light Bound

Model. For one decode step,

$$T = \frac{D + X_g}{\eta_g B_g} + \frac{X_c}{\eta_c B_c} + n_c \tau_e + \frac{X_f}{\eta_p B_p} + n_f \tau_x + n_h \tau_h, \quad (1)$$

with D the dense per-token bytes (attention, routers, norms, LM head, KV reads), X_g, X_c, X_f the routed-expert bytes executed from GPU memory, executed on the CPU (n_c executions) and fetched over PCIe (n_f transfers), B the datasheet bandwidths, and τ fixed latencies per CPU expert, transfer and hand-off. Byte counts are exact (every parameter classified on PyTorch’s **meta** device, or read from GGUF headers).

Third-party validation. On 52 published measurements from 9 independent sources (CPU-only, static of-fload in four engines, all-in-VRAM, and demand fetch; 8 models), leave-one-source-out cross-validation gives 16% median error (MAPE 17%). A per-CPU-expert latency is the one term beyond bytes over bandwidth that the data support (Table 1).

Bound. Consider any policy that keeps at most c of each layer’s experts on the GPU at any time, static or dynamic, demand-driven or prefetching. Each routed-expert execution runs on the GPU from a resident copy (cost a) or on the CPU (cost b); each resident copy needs one PCIe load (cost p), overlappable with anything. Grouping each expert’s requests into maximal runs served by one copy, a run of r requests needs one load and bridges $r-1$ reuse gaps; MIN with bypass maximises bridged gaps under the capacity limit, and a prefetching schedule bridges no more gaps than some demand schedule with bypass. Hence CPU executions $\geq M^* - \text{loads}$, with M^* the MIN-bypass misses. With x loads and m_c CPU executions per layer

step, a step costs at least $\max((k - m_c)a, m_cb)$ per layer when CPU and GPU run concurrently, convex in m_c , so by Jensen’s inequality

$$T \geq \min_{x \geq 0} \max \left(T_D + L \min_{m_c \geq (M^*/N - x)^+} \ell(m_c), Lxp \right). \quad (2)$$

Hand-offs, per-transfer latencies and the capacity cost of prefetching are dropped, so the bound stays valid. A copy pays off only if the expert is then reused $r^* = p/(b - a)$ times while resident.

5 First-Party Measurements: What the Model Gets Right

We pre-registered predictions for an NVIDIA A10 (24 GB, PCIe 4, a 30-vCPU Xeon 8358 VM) before measuring it, using only constants fitted on third-party data, and swept llama.cpp’s layer offload on gpt-oss-20b and -120b (MXFP4) and Qwen3-30B-A3B (Q4_K_M and Q8_0) with clocks locked. The platform measured 514 GB/s device reads, 151 GB/s host reads with 30 threads and 25.2 GB/s pinned host-to-device copies.

The structure holds. For every model, time per token is affine in the number of CPU layers (R^2 from 0.9965 to 0.9987), as the model requires for uniform layers, and no configuration beats the roofline. For gpt-oss the slope per CPU layer was predicted within 1% and 10%.

The transferred constants do not. The median error of the pre-registered throughput predictions was 27% (threshold 20%): GPU efficiency on the A10 is higher than on the third-party rigs (intercepts under-predicted by 25–55%), and Qwen3’s small experts run on this CPU at twice the predicted speed (slopes over-predicted by 72%). Counted DRAM bytes exceeded the GGUF-exact prediction by 12–15%, consistent with ECC and read granularity. Two calibration runs per model suffice to fix the constants (4.9% median error on the rest of the sweep, exploratory). We therefore fit constants on-platform from here on, and pre-register only what is then predicted.

6 Where the Time Goes in Hybrid Decode

We built the obvious design into llama.cpp (details in Section 7): per layer, C GPU slots hold experts; the graph splits each layer’s selected experts into resident ones, run on the GPU from their slots, and the rest, run on the CPU from pinned host memory, and adds the two results; after each step a policy copies promising misses into slots. The simulator predicted its hit rates to within 0.4 points, yet it decoded no faster than all experts on the CPU. A per-configuration regression over a diagnostic job (gpt-oss-20b, 25% budget) and targeted ablations located three costs, none of which is bandwidth.

- **Copies queue on the copy engine.** Slot copies issued with `cudaMemcpyAsync` on a side stream share the copy engine with the step’s own small host $\leftrightarrow$ device transfers, which then wait behind 13 MB expert copies: each admission cost its full PCIe time (0.48 ms) on the critical path. A copy kernel that reads pinned memory through the SMs leaves the copy engine to the step: 57.5 $\rightarrow$ 67.4 tok/s, identical logits.
- **The hand-off costs 84 μ s per layer.** With every expert resident (no CPU work at all) the split-graph cache ran at 106 tok/s against 136 for stock all-GPU: each layer that *might* need the CPU pays a stream synchronisation, two device-to-host and one host-to-device copy, a CPU graph launch over empty inputs and extra GPU launches.
- **Overlap never happened.** llama.cpp’s scheduler can run a CPU split beside a GPU split only if the former reads nothing the latter produces. Views of the layer input placed in the GPU split made every CPU split look dependent; after ignoring view nodes, 24 576 of 24 627 CPU splits ran concurrently, worth 8–13%.

With the first and third fixed, the split-graph cache was 1.11–1.39 $\times$ llama.cpp at equal memory (exploratory, four models). The second cost motivates the mailbox.

7 System

Placement and cache. Experts live in pinned host memory. Each MoE layer has C GPU slots (optionally 0 or E per layer, which reproduces llama.cpp’s layer layout). Per decode step the router’s selection is mapped, on the GPU, through two small tables (expert $\rightarrow$ slot, expert $\rightarrow$ CPU) into GPU and CPU expert ids; a negative id yields a zero row in both paths, and the results are summed. The tables are a graph input rewritten every step, so the CUDA graph is reused.

Admission. Decayed-frequency admission (DFA) keeps an exponentially decayed request count per expert (half-life 16 steps); the victim is the resident expert with the lowest count not selected this step, and a miss is admitted only if its count exceeds the victim’s by a margin κ (1 for gpt-oss, 2 for Qwen3, set on exploratory runs; it cuts copies 2–3 $\times$). Copies are issued after a step by a kernel reading pinned memory, overlap the next step and are published two steps after issue, so a slot is never read while being written.

Mailbox hand-off. In decode, a layer whose selection is not wholly resident is bracketed by two GPU operations. REQUEST checks the CPU ids on the device; if any is set, it writes the layer input and ids into a mailbox in pinned memory and publishes a sequence number (per-writer

`__threadfence_system`, then the flag). The GPU then computes the resident experts. `WAIT` spins *on the GPU* until the helpers have answered that sequence number and merges their rows (read with cache-bypassing loads) with the GPU’s. If no expert of the layer is on the CPU, both operations return immediately: no host synchronisation, copy or launch happens for that layer. Both are ordinary graph nodes, so the whole decode step remains one CUDA graph; a device-side counter makes the sequence numbers correct under graph replay, and a timeout turns a lost answer into an error rather than a hang.

Helpers. 28 threads spin on the mailbox during decode and sleep during prefill. A request runs in four phases (quantise the input, gate/up rows with bias and activation, quantise the activations, down rows written straight into the mailbox), using ggml’s own CPU kernels. Each phase is cut into chunks that helpers claim through a counter tagged with the request’s sequence number. This matters: a static split let one descheduled helper stall every request (12% run-to-run variation on a 30-vCPU VM); with claimed chunks a stalled helper delays only a chunk it holds, and repeats agree within 2%. For MXFP4 we add an AVX-512 VNNI row kernel on ggml’s block layout (1.09× ggml’s AVX2 kernel at 28 threads; the machine’s helpers then reach 107 of 143 GB/s of read bandwidth).

Exactness. With ggml’s kernels the helpers compute exactly what llama.cpp’s CPU path computes: top-5 logits of the mailbox and split-graph caches, and of all experts on the helpers and on llama.cpp’s CPU path, were bit-identical (exploratory job 016). The AVX-512 kernel changes only the float summation order. Every GPU/CPU id the graph used was checked against the host tables on-device (0 inconsistencies).

8 Pre-Registered Evaluation

Protocol. Before the test run we committed the system, the protocol, five hypotheses and quantitative predictions. Calibration used only profile sequences (2 per domain); the test uses the 12 test sequences that no exploratory run touched, each ≤ 128 prompt tokens (untimed prefill) then 192 teacher-forced decode steps, run back to back on one A10 instance with locked clocks. At each budget (12.5, 25 and 50% of each layer’s experts; gpt-oss-120b only 12.5 and 25%, as 50% does not fit) we run, at equal expert memory: llama.cpp with `--n-cpu-moe n = L - Lq`; the same layer layout on the mailbox helpers (hand-off only); the split-graph cache; and the mailbox cache.

Speed (H11, H12). Table 2 and Fig. 1 show the result. The mailbox cache is 1.41–1.81× faster than llama.cpp at every budget on every model; at the pre-registered 25% budget it is 1.63× (gpt-oss-20b), 1.52× (Qwen3 Q4_K_M), 1.50× (Q8_0) and 1.81× (gpt-oss-120b, 72.1 vs 39.8 tok/s

on a 24 GB GPU). H11 ($\geq 1.30\times$ on each model) holds. With no cache at all, moving llama.cpp’s CPU layers onto the mailbox helpers is 1.02–1.32× faster than llama.cpp. H12 ($\geq 1.05\times$ everywhere) does not hold: on Qwen3 Q8_0 the gain is only 1.022× at 12.5%, 1.047× at 25.0%. The split-graph cache, i.e. the same cache and policy behind llama.cpp’s scheduler, sits between the two; at identical hit rates the mailbox cache is 1.13–1.24× faster, which is the hand-off’s share of the gain.

Baseline check (deviation). llama.cpp warns that CPU tensor overrides are slow with a memory-mapped model and advises loading weights into buffers. Re-running every llama.cpp configuration that way (job 021, after the test run) changed its speed by -13.4% to +1.7%; the speed-ups against the faster of the two llama.cpp runs are 1.41–1.81×.

Fidelity (H13). Teacher-forced NLL of the mailbox cache differs from llama.cpp’s by at most 0.6% (relative). Its greedy top-1 agrees with llama.cpp’s on 95.6–98.7% of steps for gpt-oss-20b and Qwen3, the same level at which llama.cpp with all experts on the GPU agrees with llama.cpp’s own layer offload (95.4–98.4%): GPU and CPU kernels quantise activations differently (q8.1 vs q8.0/q8.K), and layer offload already mixes the two. For gpt-oss-120b the agreement is only 89.5–90.1%, below the registered 95%, so H13 does not hold. On this text gpt-oss-120b’s NLL is 7.36 nats (near-flat predictions, where small numeric differences flip the argmax); on its own text (below) its NLL is 0.65.

Prediction (H14). The step-time model $T = T_0 + a_0 r + a_1 \mu + \beta \alpha$ (hand-offs r , CPU experts μ and admissions α per step; llama.cpp affine in n), fitted on the calibration run and fed with per-step counts that the policy simulator produced from the routing traces of the test sequences, predicted the 33 test configurations with 2.9% median and 14.7% maximum error. H14 ($\leq 10\%$ and $\leq 20\%$) holds. The fitted a_1 is 122 μ s per gpt-oss expert (13.25 MB, i.e. 109 GB/s on the helpers, 72% of the host’s read bandwidth) and 26 μ s per Qwen3 Q4_K_M expert.

Against the bound. For the two models whose all-GPU step we measured, we evaluate the bound with first-party constants: GPU efficiency set so that the model reproduces the measured all-GPU step, CPU experts at the 151 GB/s read bandwidth with no latency, copies at 25.2 GB/s, and MIN-bypass misses from the test traces. It is 181 and 197 tok/s and barely depends on the budget: when CPU and GPU run concurrently, the optimum gives the CPU a share of every layer’s experts, so the bound exceeds all-GPU speed (139 and 147 tok/s). llama.cpp reaches 30–42% of it and the mailbox cache 46–63%. The same accounting applied to the misses DFA actually incurs, with no overheads, gives the speed an ideal implementation

Table 2: Test run (12 fresh sequences $\times$ 192 decode steps per configuration, A10). Decode tok/s at equal GPU expert memory. “helpers” is llama.cpp’s layer layout with CPU layers on the mailbox helpers. Speed-up and predicted are for the mailbox cache; predicted is the pre-registered value. NLL Δ and top-1 compare the mailbox cache with llama.cpp at the same budget.

model	budget	llama.cpp	helpers	split cache	mailbox cache	speed-up	predicted	hit	NLL Δ	top-1
gpt-oss-20b	12.5%	54.5	65.3	67.5	83.0	1.52 $\times$	83.6	0.476	-0.04%	95.9%
	25.0%	59.9	70.9	79.8	97.4	1.63 $\times$	96.7	0.682	-0.00%	95.6%
	50.0%	71.5	85.8	97.4	114.6	1.60 $\times$	117.1	0.882	+0.08%	95.6%
	all on GPU	138.6								
Qwen3 Q4_K_M	12.5%	62.7	71.1	75.1	91.6	1.46 $\times$	95.9	0.600	+0.00%	98.4%
	25.0%	68.1	77.1	85.1	103.6	1.52 $\times$	106.9	0.772	-0.15%	98.7%
	50.0%	82.1	92.0	93.8	116.2	1.41 $\times$	125.3	0.909	-0.10%	98.6%
	all on GPU	147.2								
Qwen3 Q8_0	12.5%	48.1	49.1	58.1	67.6	1.41 $\times$	67.4	0.601	-0.05%	98.7%
	25.0%	52.2	54.6	68.4	78.5	1.50 $\times$	77.7	0.772	+0.06%	98.4%
	50.0%	61.5	66.2	75.5	88.9	1.45 $\times$	91.7	0.909	-0.06%	98.4%
gpt-oss-120b	12.5%	41.5	47.9	56.4	65.0	1.57 $\times$	60.6	0.626	-0.24%	90.1%
	25.0%	39.8	52.5	63.6	72.1	1.81 $\times$	67.7	0.783	-0.56%	89.5%

of this policy would reach; the mailbox cache achieves 58–80% of it. The remaining gap therefore has two parts of similar size: implementation (helpers at 72% of read bandwidth, per-request synchronisation, copy interference) and policy (MIN-bypass incurs 46–59% fewer misses than DFA at 25%).

The models’ own text (H15). Teacher forcing on dataset responses is off-policy. We therefore sampled 193-token continuations of the 12 test prompts from each model (top- k 40, top- p 0.95, temperature 0.8, fixed seeds, generated by llama.cpp at the 25% layout), then teacher-forced every system on that text. At 25% the mailbox cache is 1.54–1.94 $\times$ llama.cpp (H15, $\geq 1.30\times$: holds), and gpt-oss-120b’s NLL on its own text is 0.65 nats against 7.36 on the dataset text: its routing there is that of a model predicting well.

9 Discussion

For engine builders. The largest single cost we found was not bandwidth but coordination: a host round trip per layer. A GPU-signalled hand-off is independent of caching and would speed up llama.cpp’s existing layer offload by itself (H12). CPU helpers must tolerate preemption; on a VM with every vCPU busy, static work splits do not.

Where the rest of the gap is. At 25% the mailbox cache’s step is the all-GPU step plus, per layer with CPU experts, the helpers’ time for those experts minus the GPU work they replace; the fitted a_1 corresponds to 72% of the host read bandwidth. The remaining levers are fewer misses (the MIN-bypass gap, which needs prediction of the next token’s routing, not only of the next layer’s), and PCIe as a third concurrent source: streaming a predicted

miss costs $p \approx 0.53$ ms for a gpt-oss expert, so on this platform the link could carry about half of today’s CPU experts per step. Per-layer slot allocation, in contrast, removed at most 5% of misses in simulation.

For evaluation. Hit rate did not predict speed (the first cache matched the simulator’s hit rates and was no faster than no cache); per-step counts of hand-offs, CPU experts and copies did. Reporting speed as a fraction of a platform-specific bound, with the baseline users actually run at equal memory, makes such claims comparable.

10 Limitations

- **One platform.** All first-party results are from one A10 VM (PCIe 4, one socket). The mailbox relies on pinned host memory mapped into the GPU’s address space and on the GPU observing host writes, which CUDA supports on x86 hosts; other interconnects and multi-socket hosts are untested.
- **Batch 1, decode only.** Prefill uses llama.cpp’s existing path. Multi-request serving takes the union of experts and weakens locality.
- **Busy CPU.** The helpers spin on 28 of 30 vCPUs during decode, which suits a dedicated inference box but not a shared desktop; parking them costs wake-up latency.
- **Off-policy traces.** The simulator’s inputs are dataset text; H15 checks the speed-up on the models’ own text, but prediction there is not evaluated.
- **Scope of the bound.** It covers per-layer capacity; a cross-layer pool could do better.

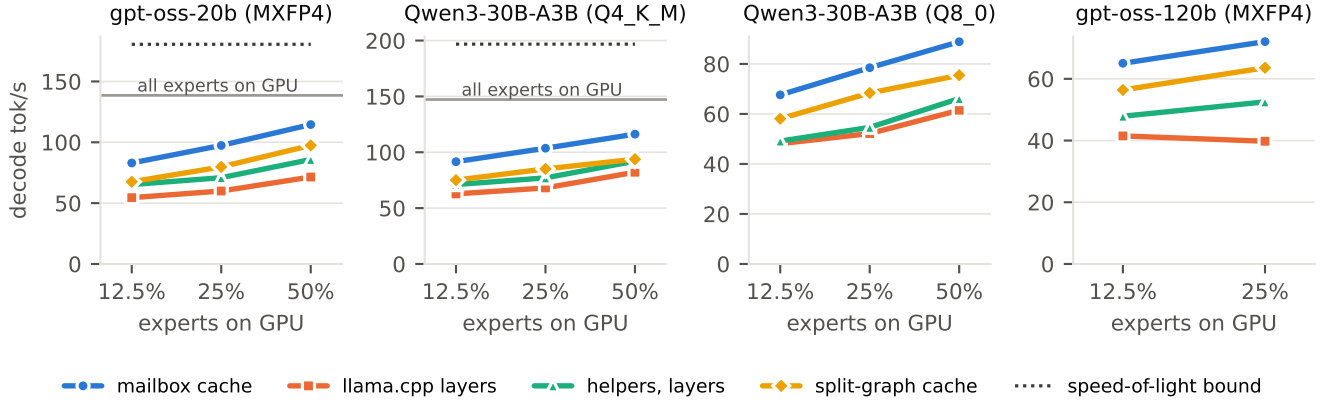


Figure 1: Test run: decode tok/s against the share of each layer’s experts held on the GPU (equal expert memory for all systems). Dotted: speed-of-light bound (models with a measured all-GPU step); grey line: all experts on the GPU.

11 Conclusion

Exact routing traces, a validated bytes-over-bandwidth model and a speed-of-light bound say how fast MoE decode with experts in host memory could be. Measuring the engine people use showed that most of the gap was coordination, not bandwidth. An expert cache whose misses run on CPU helps that the GPU signals directly closes much of it: $1.41\text{--}1.81\times$ over llama.cpp at equal memory on four models in a pre-registered test, with unchanged outputs, predicted in advance by the model.

Acknowledgments. Parts of the code, the experiments and the text were drafted with the help of an AI system (Claude, Anthropic). The author reviewed the work and takes responsibility for its content.

References

- [1] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [2] Pei Cao, Edward W. Felten, Anna R. Karlin, and Kai Li. A study of integrated prefetching and caching strategies. In *Proceedings of ACM SIGMETRICS*, 1995.
- [3] Shiyi Cao, Shu Liu, et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. *arXiv preprint arXiv:2411.11217*, 2024.
- [4] Gil Einziger, Roy Friedman, and Ben Manes. TinyLFU: A highly efficient cache admission policy. *ACM Transactions on Storage*, 13(4), 2017.
- [5] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [6] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023–2026.
- [7] Torsten Hoeffler, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- [8] Torsten Hoeffler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [9] En-Ming Huang, Li-Shang Lin, et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- [10] Ranggi Hwang, Jianyu Wei, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. *arXiv preprint arXiv:2308.12066*, 2023.
- [11] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoeffler. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [12] Keisuke Kamahori, Tian Tang, et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [13] KVCache.AI and contributors. KTransformers: A flexible framework for CPU/GPU hybrid LLM inference. <https://github.com/kvcache-ai/ktransformers>, 2024–2026. See also the SOSP 2025 paper by the same team.
- [14] Yuan Si, Yufeng Lin, et al. Who should own the expert cache? kernel-managed tiering for trillion-parameter MoE inference. *arXiv preprint arXiv:2608.12103*, 2026.
- [15] Xiaoni Song, Zihang Zhong, et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.
- [16] Peng Tang, Jiacheng Liu, et al. HOBBIT: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [17] Wenxin Wang, Yule Hou, et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-

- GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026.
- [18] Zihan Wang, Yuqi Wang, et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026.
 - [19] Leyang Xue, Yao Fu, et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
 - [20] Yichao Yuan, Lin Ma, et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
 - [21] Yu Zhang. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026.
 - [22] Yushu Zhao, Yubin Qin, et al. MoBiLE: Efficient mixture-of-experts inference on consumer GPU with mixture of big little experts. *arXiv preprint arXiv:2510.12357*, 2025.
 - [23] Shuzhang Zhong et al. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *Design Automation Conference (DAC)*, 2025. arXiv:2504.05897.
 - [24] Shien Zhu, Samuel Bohl, et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.